



Red Hat OpenShift AI Self-Managed 3.0

Creating a workbench

Create a workbench and a custom image by using Custom Resource Definitions (CRDs) and the command-line.

Red Hat OpenShift AI Self-Managed 3.0 Creating a workbench

Create a workbench and a custom image by using Custom Resource Definitions (CRDs) and the command-line.

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

As a cluster administrator, you can create a workbench and a custom image by using Custom Resource Definitions (CRDs) and the OpenShift Command Line Interface (CLI).

Table of Contents

CHAPTER 1. OVERVIEW	3
CHAPTER 2. CREATING A CUSTOM IMAGE BY USING THE IMAGESTREAM CRD	4
CHAPTER 3. CREATING A WORKBENCH BY USING THE NOTEBOOK CRD	8

CHAPTER 1. OVERVIEW

In Red Hat OpenShift AI, a workbench is an isolated area where a data scientist can examine and work with ML models. When you create a workbench, you specify a workbench image. OpenShift AI provides a selection of default workbench images that you can choose from. Each image is optimized with the tools and libraries that a data scientist needs for model development. To view a list of the OpenShift AI default workbench images and their preinstalled packages, see [Supported Configurations for 3.x](#).

As a cluster administrator, you can create a custom image, for example, if a data scientist on your team requires a specific version of a library that is different from the version provided in a default image. For information about OpenShift AI custom images, see [Creating custom workbench images](#).

You have the following options for creating workbenches and custom images:

- As an OpenShift cluster administrator, you can create a custom image and a workbench by using OpenShift AI Custom Resource Definitions (CRDs) and the OpenShift CLI (**oc**) as described in this guide.
- As an OpenShift cluster administrator, you can use OpenShift APIs to create resources, such as a custom image. You can programmatically call the APIs through HTTP GET methods in your code, a Bash script, or a Python script. For more information about using the OpenShift APIs to create an ImageStream resource, see the [ImageStream entry](#) in the *OpenShift API Reference*.
- As any OpenShift AI user, you can use the OpenShift AI dashboard to create workbenches and select images, as described in [Using project workbenches](#).

CHAPTER 2. CREATING A CUSTOM IMAGE BY USING THE IMAGESTREAM CRD

You can create a custom image by using the **ImageStream** Custom Resource Definition (CRD).

In the following procedure, you configure an **ImageStream** CRD and use it to create the **ImageStream** Custom Resource (CR) that defines the custom image. The **ImageStream** CR provides a URL for the custom image, which you need when you want to use the custom image to configure a workbench.

Note: The custom image that you create also becomes available in the OpenShift AI dashboard so that your data scientist users can select it when they create a workbench.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in as shown in the following example:

```
oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Define the **ImageStream** CRD.
 - a. Create a YAML manifest file named **notebook-image-stream.yaml**.
 - b. Copy the following configuration and paste it in the **notebook-image-stream.yaml** file:

Example ImageStream

```
kind: ImageStream
apiVersion: image.openshift.io/v1
metadata:
  annotations:
    opendatahub.io/notebook-image-desc: A custom Jupyter Notebook image ❶
    opendatahub.io/notebook-image-name: My Custom Notebook ❷
  name: my-custom-notebook
  namespace: redhat-ods-applications ❸
  labels: ❹
    app.kubernetes.io/created-by: byon
    opendatahub.io/dashboard: 'true'
    opendatahub.io/notebook-image: 'true'
    opendatahub.io/component: 'true'
    platform.opendatahub.io/part-of: 'workbenches'
    app.kubernetes.io/part-of: 'workbenches'
spec:
```



```

lookupPolicy:
  local: true
tags:
  - name: '1.0' 5
    annotations: 6
      opendatahub.io/notebook-python-dependencies:
        '[{"name":"PyTorch","version":"2.2"}]' 7
      opendatahub.io/notebook-software: '[{"name":"Python","version":"v3.11"}]' 8
      opendatahub.io/workbench-image-recommended: 'true' 9
      opendatahub.io/image-tag-outdated: 'true' 10
      opendatahub.io/notebook-build-commit: '3e71410' 11
from:
  kind: DockerImage
  name: 'quay.io/modh/rocm-
notebooks@sha256:199367d2946..b411433ffbb5f0988279b10150020af22db' 12
importPolicy:
importPolicy:
  importMode: Legacy
referencePolicy:
  type: Source

```

The example YAML file includes the following information:

- 1 A description of the image.
- 2 The image name that is displayed in the drop down menu when a user creates a workbench in the OpenShift AI dashboard.
- 3 The **redhat-ods-applications** namespace is the default namespace in which the **ImageStream** CR is created.
- 4 The labels that are required if you want the image to appear in the OpenShift AI dashboard. The **app.kubernetes.io/created-by: byon** label identifies the origination of the image object.
- 5 Annotations that are required if you want to make the image available in the OpenShift AI dashboard.
- 6 The version for the image. You can configure multiple versions for the same image. For this example, the version is 1.0.
- 7 An annotation that gives the user information about the Python packages and versions that are pre-installed in the image.
- 8 An annotation that specifies information such as the Python version, Jupyter version, or CUDA version.
- 9 An annotation that specifies whether the **ImageStream** version is the default version of the image. Set this field to **'true'** if the **ImageStream** version is the default. Otherwise, set it to **'false'**. You must specify the **opendatahub.io/workbench-image-recommended** annotation field if there are multiple versions of the image with different configurations. If you have only one version of the image, set the field to **'true'**.
- 10 An annotation that specifies whether the image version has tags that are outdated and

out of the regular maintenance cycle.

- 11 An annotation that references the commit hash's build commit ID to identify the sources that the specific tag was built from.
- 12 The image registry path where the image has been uploaded.

- To create the **ImageStream** CR, run the following command, where the **ImageStream** CRD YAML manifest file name is **notebook-image-stream.yaml**:

```
oc create -f notebook-image-stream.yaml
```

Verification

- To verify that the **ImageStream** was successfully created, run the following command, where the name of the **ImageStream** is **my-custom-notebook**:

```
oc describe imagestream my-custom-notebook -n redhat-ods-applications
```

You should see output similar to the following example:

Example output

```
Name: my-custom-notebook
Namespace: redhat-ods-applications
Created: 6 minutes ago
Labels: app.kubernetes.io/created-by=byon
        opendatahub.io/dashboard=true
        opendatahub.io/notebook-image=true
Annotations: kubectl.kubernetes.io/last-applied-configuration=
{"apiVersion":"image.openshift.io/v1","kind":"ImageStream","metadata":{"annotations":
{"opendatahub.io/notebook-image-desc":"A custom Jupyter Notebook
image","opendatahub.io/notebook-image-name":"My Custom Notebook"},"labels":
{"app.kubernetes.io/created-
by":"byon","opendatahub.io/dashboard":"true","opendatahub.io/notebook-
image":"true"},"name":"my-custom-notebook","namespace":"redhat-ods-applications"},"spec":
{"lookupPolicy":{"local":true},"tags":[{"annotations":{"opendatahub.io/notebook-python-
dependencies":[{"name":"PyTorch","version":"2.2"}],"opendatahub.io/notebook-
software":[{"name":"Python","version":"v3.11"}],"opendatahub.io/workbench-image-
recommended":true},"from":{"kind":"DockerImage","name":"quay.io/modh/rocm-
notebooks@sha256:199367d2946fc8...8279b10150020af22db"},"importPolicy":
{"importMode":"Legacy"},"name":"1.0","referencePolicy":{"type":"Source"}}]}
opendatahub.io/notebook-image-desc=A custom Jupyter Notebook image
opendatahub.io/notebook-image-name=My Custom Notebook
openshift.io/image.dockerRepositoryCheck=2025-03-10T11:02:44Z
Image Repository: image-registry.openshift-image-registry.svc:5000/redhat-ods-
applications/my-custom-notebook
Image Lookup: local=true
Unique Images: 1
Tags: 1
1.0
tagged from quay.io/modh/rocm-
```

```
notebooks@sha256:199367d2946..b411433ffbb5f0988279b10150020af22db
```

```
* quay.io/modh/rocm-
notebooks@sha256:199367d2946fc8427....1433ffbb5f0988279b10150020af22db
  6 minutes ago
```

2. To determine the URL for your custom image so that you can reference it when you create a workbench:

- a. Make a note of the values for the **Image Repository** and the **Tags** fields from the **ImageStream** output.

In the following example, the **Image Repository** value is **image-registry.openshift-image-registry.svc:5000/redhat-ods-applications/my-custom-notebook** and the **Tags** value is **1.0**:

Example output

```
....
Image Repository:   image-registry.openshift-image-registry.svc:5000/redhat-ods-
applications/my-custom-notebook
Image Lookup:       local=true
Unique Images:      1
Tags:               1
1.0
  tagged from quay.io/modh/rocm-
notebooks@sha256:199367d2946..b411433ffbb5f0988279b10150020af22db
....
```

- b. Create a fully-formed image URL by combining the values for the **Image Repository** and the **Tags** fields, as shown in the following example:

```
image-registry.openshift-image-registry.svc:5000/redhat-ods-applications/my-custom-
notebook:1.0
```

CHAPTER 3. CREATING A WORKBENCH BY USING THE NOTEBOOK CRD

In OpenShift AI, you can create a workbench object by using the **Notebook** Custom Resource Definition (CRD).

In the following procedure, you configure a **Notebook** CRD and then use it to create the **Notebook** Custom Resource (CR) that defines the workbench.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have created a project. In the example in this procedure, the project is named **my-project**.
- You know the URL for the image that you want to use in the workbench. The example in this procedure uses the custom image that you created in [Creating a custom image by using the ImageStream CRD](#).

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in as shown in the following example:

```
oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Define the **Notebook** CRD.
 - a. Create a YAML manifest file named **notebook.yaml**.
 - b. Copy the following configuration and paste it in the **notebook.yaml** file:

Example Notebook

```
apiVersion: kubeflow.org/v1
kind: Notebook
metadata:
  annotations:
    notebooks.opendatahub.io/inject-oauth: 'true' ❶
   .opendatahub.io/image-display-name: My Custom Notebook ❷
    notebooks.opendatahub.io/oauth-logout-url: 'https://<dashboard_URL>/projects/my-project?notebookLogout=my-workbench'
    .opendatahub.io/accelerator-name: "
    openshift.io/description: " ❸
    openshift.io/display-name: My Workbench ❹
    notebooks.opendatahub.io/last-image-selection: 'my-custom-notebook:1.0'
    notebooks.kubeflow.org/last_activity_check_timestamp: '2024-07-30T20:43:25Z'
```

```

notebooks.opendatahub.io/last-size-selection: Small
opendatahub.io/username: 'kube:admin'
notebooks.kubeflow.org/last-activity: '2024-07-30T20:27:25Z'
opendatahub.io/connections: 'my-project/my-s3-connection'
opendatahub.io/connections: 'my-project/my-uri-connection'
opendatahub.io/connections: 'my-project/my-oci-connection'
name: my-workbench 5
namespace: my-project 6
labels:
  kueue.x-k8s.io/queue-name: <local-queue-name> 7
spec:
  template:
    spec:
      affinity: {}
      containers:
        - resources: 8
          limits:
            cpu: '2'
            memory: 8Gi
          requests:
            cpu: '1'
            memory: 8Gi
      readinessProbe:
        failureThreshold: 3
        httpGet:
          path: /notebook/my-project/my-workbench/api
          port: notebook-port
          scheme: HTTP
        initialDelaySeconds: 10
        periodSeconds: 5
        successThreshold: 1
        timeoutSeconds: 1
      name: my-workbench
      livenessProbe:
        failureThreshold: 3
        httpGet:
          path: /notebook/my-project/my-workbench/api
          port: notebook-port
          scheme: HTTP
        initialDelaySeconds: 10
        periodSeconds: 5
        successThreshold: 1
        timeoutSeconds: 1
      env: 9
        - name: NOTEBOOK_ARGS
          value: |-
            --ServerApp.port=8888
            --ServerApp.token=""
            --ServerApp.password=""
            --ServerApp.base_url=/notebook/my-project/my-workbench
            --ServerApp.quit_button=False
            --ServerApp.tornado_settings={"user": "<user>", "hub_host": "<dashboard_URL>", "hub_prefix": "/projects/my-project"}
        - name: JUPYTER_IMAGE
          value: 'image-registry.openshift-image-registry.svc:5000/redhat-ods-applications/my-custom-notebook:1.0'

```

```

- name: PIP_CERT
  value: /etc/pki/tls/custom-certs/ca-bundle.crt
- name: REQUESTS_CA_BUNDLE
  value: /etc/pki/tls/custom-certs/ca-bundle.crt
- name: SSL_CERT_FILE
  value: /etc/pki/tls/custom-certs/ca-bundle.crt
- name: PIPELINES_SSL_SA_CERTS
  value: /etc/pki/tls/custom-certs/ca-bundle.crt
- name: GIT_SSL_CAINFO
  value: /etc/pki/tls/custom-certs/ca-bundle.crt
ports:
- containerPort: 8888
  name: notebook-port
  protocol: TCP
imagePullPolicy: Always
volumeMounts:
- mountPath: /opt/app-root/src
  name: my-workbench
- mountPath: /dev/shm
  name: shm
- mountPath: /etc/pki/tls/custom-certs/ca-bundle.crt
  name: trusted-ca
  readOnly: true
  subPath: ca-bundle.crt
image: 'image-registry.openshift-image-registry.svc:5000/redhat-ods-
applications/my-custom-notebook:1.0' 10
workingDir: /opt/app-root/src
- resources: 11
  limits:
    cpu: 100m
    memory: 64Mi
  requests:
    cpu: 100m
    memory: 64Mi
readinessProbe:
  failureThreshold: 3
  httpGet:
    path: /oauth/healthz
    port: oauth-proxy
    scheme: HTTPS
  initialDelaySeconds: 5
  periodSeconds: 5
  successThreshold: 1
  timeoutSeconds: 1
name: oauth-proxy
livenessProbe:
  failureThreshold: 3
  httpGet:
    path: /oauth/healthz
    port: oauth-proxy
    scheme: HTTPS
  initialDelaySeconds: 30
  periodSeconds: 5
  successThreshold: 1
  timeoutSeconds: 1
env:

```

```

- name: NAMESPACE
  valueFrom:
    fieldRef:
      fieldPath: metadata.namespace
ports:
- containerPort: 8443
  name: oauth-proxy
  protocol: TCP
imagePullPolicy: Always
volumeMounts:
- mountPath: /etc/oauth/config
  name: oauth-config
- mountPath: /etc/tls/private
  name: tls-certificates
image: 'registry.redhat.io/openshift4/ose-oauth-proxy-
rhel9@sha256:ca21e218e26c46e3c63d926241846f8f307fd4a586cc4b04147da49af6018ef
5'
args:
- '--provider=openshift'
- '--https-address=:8443'
- '--http-address='
- '--openshift-service-account=my-workbench'
- '--cookie-secret-file=/etc/oauth/config/cookie_secret'
- '--cookie-expire=24h0m0s'
- '--tls-cert=/etc/tls/private/tls.crt'
- '--tls-key=/etc/tls/private/tls.key'
- '--upstream=http://localhost:8888'
- '--upstream-ca=/var/run/secrets/kubernetes.io/serviceaccount/ca.crt'
- '--email-domain='
- '--skip-provider-button'
- '--openshift-sar=
{"verb": "get", "resource": "notebooks", "resourceAPIGroup": "kubeflow.org", "resourceName":
"my-workbench", "namespace": "${NAMESPACE)}"
- '--logout-url=<dashboard_URL>/projects/my-project?notebookLogout=my-
workbench'
enableServiceLinks: false
serviceAccountName: my-workbench
volumes:
- name: my-workbench
  persistentVolumeClaim:
    claimName: my-workbench
- emptyDir:
    medium: Memory
    name: shm
- configMap:
    items:
      - key: ca-bundle.crt
        path: ca-bundle.crt
    name: workbench-trusted-ca-bundle
    optional: true
    name: trusted-ca
- name: oauth-config
  secret:
    defaultMode: 420
    secretName: my-workbench-oauth-config
- name: tls-certificates

```

```
secret:
  defaultMode: 420
  secretName: my-workbench-tls
```

The example YAML file includes the following information:

- 1 The **inject-oauth** annotation generates other OAUTH-based configurations, such as, the **oauth-proxy**, automatically. The default value is **true**.
 - 2 The **Notebook** image name is visible in the OpenShift AI dashboard. In this example, the image name is **My custom notebook**. Optionally, you can name the image according to your use case.
 - 3 An optional description of the workbench.
 - 4 The workbench name that is displayed in the OpenShift AI dashboard. In this example, the display name is **My Workbench**.
 - 5 The name for the workbench. In this example, the workbench name is **my-workbench**.
 - 6 The project for the workbench. In this example, the project name is **my-project**.
 - 7 To queue your workbench (Notebook) Pods and manage their resources, add the **queue.x-k8s.io/queue-name** label to the **spec.template.metadata.labels** of the **Notebook** CR. Set the value to the name of an existing **LocalQueue** in your project. This is required only if your project is enabled for Kueue.
 - 8 The deployment size for the container. You can set **limits** and **requests** values for CPU and memory.
 - 9 Environment variables for configuring values, for example, for Jupyter Notebook arguments and SSL/TLS certificates.
 - 10 The **Notebook** image. In this example, **image-registry.openshift-image-registry.svc:5000/redhat-ods-applications/my-custom-notebook:1.0** is the **Notebook** image. You can select the image version based on the packages included in the image.
 - 11 The **inject-oauth** annotation configures the **oauth-proxy** container section of the **Notebook**.
3. Edit the **notebooks.opendatahub.io/oauth-logout-url** field, annotated as (1) in the following example. Replace **my-project** with the name of the project that you created.

Example Notebook

```
apiVersion: kubeflow.org/v1
kind: Notebook
metadata:
  annotations:
    ...
    notebooks.opendatahub.io/oauth-logout-url: '<dashboard_URL>/projects/my-project?
notebookLogout=my-workbench' 1
    ...
```


4. Edit the **--logout-url=** field, annotated as **(1)** in the following example. Replace **my-project** with the name of the project that you created.

Example Notebook

```
apiVersion: kubeflow.org/v1
kind: Notebook
metadata:
  annotations:
  ...
spec:
  template:
    spec:
      affinity: {}
      containers:
      ...
      - resources:
      ...
      args:
      - '--logout-url=<dashboard_URL>/projects/my-project?notebookLogout=my-workbench'
(1)
  ...
```

5. Edit the **value** field of the **JUPYTER_IMAGE** environment variable, annotated as **(1)** in the following example. Replace the image URL with the URL of the custom image that you created.

Example Notebook

```
apiVersion: kubeflow.org/v1
kind: Notebook
metadata:
  annotations:
  ...
spec:
  template:
    spec:
      affinity: {}
      containers:
      ...
      - resources:
      ...
      env:
      ...
      - name: JUPYTER_IMAGE
        value: 'image-registry.openshift-image-registry.svc:5000/redhat-ods-applications/my-
custom-notebook:1.0' (1)
```

6. Edit the **image** field, annotated as **(1)** in the following example. Replace the image URL with the URL of the custom image that you created.

Example Notebook

```
apiVersion: kubeflow.org/v1
kind: Notebook
```

```

metadata:
  annotations:
  ...
spec:
  template:
    spec:
      affinity: {}
      containers:
      ...
      - resources:
      ...
      env:
      ...
      ports:
      ..
      imagePullPolicy: Always
      volumeMounts:
      ...
      image: 'image-registry.openshift-image-registry.svc:5000/redhat-ods-applications/my-
custom-notebook:1.0' 1
      workingDir: /opt/app-root/src
      ...

```

7. To create the **Notebook** CR, run the following command, where the **Notebook** CRD YAML manifest filename is **notebook.yaml**.

```
oc create -f notebook.yaml
```

Verification

- To verify that the workbench was successfully created, run the following command, replacing **my-project** with the name of the project where you created the **Notebook** CR.

```
oc describe notebook -n my-project
```

You should see output similar to the following example:

Example output

```

Name:      my-workbench
Namespace: my-project
Labels:    <none>
Annotations: notebooks.kubeflow.org/last-activity: 2024-07-30T20:27:25Z
             notebooks.kubeflow.org/last_activity_check_timestamp: 2024-07-30T20:43:25Z
             notebooks.opendatahub.io/inject-oauth: true
             notebooks.opendatahub.io/last-image-selection: my-custom-notebook:1.0
             notebooks.opendatahub.io/last-size-selection: Small
             notebooks.opendatahub.io/oauth-logout-url:
               <dashboard_URL>/projects/my-project?notebookLogout=my-workbench
            .opendatahub.io/accelerator-name:
             .opendatahub.io/image-display-name: My Custom Notebook
             .opendatahub.io/username: kube:admin
             .openshift.io/description:
             .openshift.io/display-name: My Workbench

```

```

API Version: kubeflow.org/v1
Kind:      Notebook
Metadata:
  Creation Timestamp: 2025-03-06T13:27:25Z
  Generation:        1
  Resource Version:   42316914
  UID:                89f4....9e9-7c48-4f53-9397-05c....d21a
Spec:
  Template:
    Spec:
      Affinity:
      Containers:
        Env:
          Name: NOTEBOOK_ARGS
          Value: --ServerApp.port=8888
        --ServerApp.token=""
        --ServerApp.password=""
        --ServerApp.base_url=/notebook/my-project/my-workbench
        --ServerApp.quit_button=False
        --ServerApp.tornado_settings={"user":"kube-3aadmin","hub_host":"<dashboard_URL>",
        "hub_prefix":"/projects/my-project"}
          Name: JUPYTER_IMAGE
          Value: image-registry.openshift-image-registry.svc:5000/redhat-ods-
applications/my-custom-notebook:1.0
          Name: PIP_CERT
          Value: /etc/pki/tls/custom-certs/ca-bundle.crt
          Name: REQUESTS_CA_BUNDLE
          Value: /etc/pki/tls/custom-certs/ca-bundle.crt
          Name: SSL_CERT_FILE
          Value: /etc/pki/tls/custom-certs/ca-bundle.crt
          Name: PIPELINES_SSL_SA_CERTS
          Value: /etc/pki/tls/custom-certs/ca-bundle.crt
          Name: GIT_SSL_CAINFO
          Value: /etc/pki/tls/custom-certs/ca-bundle.crt
          Image: image-registry.openshift-image-registry.svc:5000/redhat-ods-
applications/my-custom-notebook:1.0
        Image Pull Policy: Always
        Liveness Probe:
          Failure Threshold: 3
          Http Get:
            Path: /notebook/my-project/my-workbench/api
            Port: notebook-port
            Scheme: HTTP
          Initial Delay Seconds: 10
          Period Seconds: 5
          Success Threshold: 1
          Timeout Seconds: 1
        Name: my-workbench
        Ports:
          Container Port: 8888
          Name: notebook-port
          Protocol: TCP
        Readiness Probe:
          Failure Threshold: 3
          Http Get:
            Path: /notebook/my-project/my-workbench/api

```

```
Port:          notebook-port
Scheme:        HTTP
Initial Delay Seconds: 10
Period Seconds: 5
Success Threshold: 1
Timeout Seconds: 1
Resources:
Limits:
  Cpu: 2
  Memory: 8Gi
Requests:
  Cpu: 1
  Memory: 8Gi
Volume Mounts:
Mount Path: /opt/app-root/src
Name:      my-workbench
Mount Path: /dev/shm
Name:      shm
Mount Path: /etc/pki/tls/custom-certs/ca-bundle.crt
Name:      trusted-ca
Read Only: true
Sub Path:  ca-bundle.crt
Working Dir: /opt/app-root/src
Args:
  --provider=openshift
```